

Quantum Approximate Optimization Algorithm (QAOA) Implementation for Weighted & Non-Weighted Max-Cut Problems

Abstract

The Quantum Approximate Optimization Algorithm (QAOA) tries to solve optimization problems by turning them into a quantum “energy minimization” problem with bitstring solutions. With applications in physics, finance, machine learning, and other fields, the QAOA is useful for combinatorial problems where brute-force iteration through possible solutions is computationally intractable as problem size scales.

The Max-Cut problem has an undirected graph with vertices and edges, where the goal is to separate the vertices into two groups so that the maximum number of edges is “cut”. Similarly, this problem can be adjusted to accommodate different edge weights, changing the question to how to cut through edges to maximize the sum of cut edge weights. As the number of vertices increases, the possible number of solutions to Max-Cut increases exponentially, making it an NP-Hard problem to solve through brute force. This project explores the Quantum Approximate Optimization Algorithm (QAOA) for both weighted and non-weighted Max-Cut scenarios against brute force methods and tests the algorithm layer-depth variable p in improving results.

Procedure

Implementing the QAOA is dependent on whether there is a possible way to encode a cost Hamiltonian for a particular problem. The cost Hamiltonian encodes what is a “good” and a “bad” answer within a phase, and when combined with a mixer Hamiltonian, which allows us to spread the probability amplitudes between bitstring answers, produces interference within the quantum state that amplifies the “good” answers while suppressing the “bad” answers. Thankfully, for the unweighted and weighted Max-Cut, the cost hamiltonians are already known, with a slight variation in the latter to account for weights. The idea is that our “optimal solution” is the ground state, so through QAOA, we try to get our quantum state closest to the ground state, by minimizing energy, we maximize max cuts. For the Max-Cut, solutions will be represented as tuples with each index corresponding to a vertex in a given graph, where all vertices with bit 0 are within the same group, and likewise.

First, I defined functions to create the cost and mixer Hamiltonians based on the known formulas, then optimized the parameters gamma and beta to minimize the energy given by taking the expectation value of the Hamiltonians. From there, we apply the cost, then mixer Hamiltonians a p number of times to get a post-QAOA quantum state. To get a probability distribution of the different bitstring solutions, we conduct random sampling and then analyze the top max cut values and their respective frequencies in the probability distribution. For a comparison and to know the truly exact answers, I also implemented a method that brute-force searches through all possible bitstrings for the solutions that produce the maximum cut value. For the unweighted Max-Cut, I made 4 arbitrary test cases, and for the weighted, I made 3 to analyze.

Ideally, as p , the number of QAOA layers, increases, we'd get better results, as with more layers, the probability distribution is supposed to get closer to the "optimal solutions" or max cut value. To test if this is the case, I implemented QAOA for a range of $[1, 3]$ and plotted the best cut value from brute force, the best cut value from the QAOA, and the average cut value for a basic unweighted test case, as well as the relationship of the probability of obtaining the max cut value to an increasing p .

Results

Note: There are differences in the results compared to my final presentation. When double-checking my code before the final submission, I realized why my results' cut frequencies and the analysis of p were off; I forgot to put a negative sign on a line of code... As such, this report will reflect these newfound better results, and I will attach the new data and plots as extra slides in the presentation.

When comparing my QAOA results to the brute force solutions for both unweighted and weighted Max-Cut, we see that for all test cases, they're the same. Notably, for most cases, the solutions with the maximum cut values have high frequencies in the probability distribution. When drawing out the "line" that separates the vertices into two groups of nodes on the graph visualization, we also see that the groupings and cut values are accurate. The energy given from QAOA is ideally the negative of the maximum cut value, and I got close numbers for the unweighted test cases, with the best example being the diamond graph with an energy of 3.697 compared to the max cut of 4 and a worst for the star graph with energy 2.993 compared to max cut 4. Similarly with the unweighted, there were varied QAOA energy results with larger differences due to larger cut numbers. The best was the heavily weighted graph with an energy of 11.520 compared to the max cut of 12, and the worst being the triangle and rectangle graph with an energy of 9.209 compared to its max cut of 12. Despite these energies being off, the QAOA was still able to find the best cut values with high frequency.

In my analysis of the p variable (QAOA layer depth), we see that through my optimization method, the average cut value increases as p increases, meaning the probability density from the post-QAOA state is moving closer to the optimal cut value, and the QAOA found max cut value is equal to the value found through brute force. In the probability of optimal value vs. p plot, generally, we witness a positive trend, meaning that with more layers, the probability of getting the right solutions is increasing. Notably, with both plots, due to the random sampling nature of the QAOA, there can be slight differences with every run of the code, but generally, these are the trends seen.

Conclusion

The QAOA works well with graphs of vertices 4-5 in achieving the same results as brute forcing through possible solutions. For my particular test cases, brute force performs better in

solutions and runtime; however, for graphs with a larger number of vertices, this shouldn't be the case.

My original conclusion, as presented in class, that my optimization method was not exact enough to demonstrate that a larger p is supposed to produce better results by increasing the application of hamiltonians that amplify the “good cuts”, was incorrect due to finding the bug in my code. With the fix, my optimization method to SciPy's is quite comparable with positive linear line plots, and the ideal relationship is shown. However, due to that mistake, I learned that the parameters increase by two per layer, meaning that with more parameters, optimization becomes more difficult. This also means that as the number of vertices increases, increasing the quantum state dimensions, these parameters are given the difficult task of controlling complex quantum states, making optimization even more difficult. Future extensions of this project would be to explore more complicated graphs for Max-Cut and other applications of QAOA in NP-hard problems like portfolio optimization.